
PROJECT REPORT: MASS EMAILING USING AWS LAMBDA

Course: Cloud AI with AWS Internship

Organization: Cloudcredits Technologies Pvt. Ltd.

Student Name: OM MOAHN CHAVAN

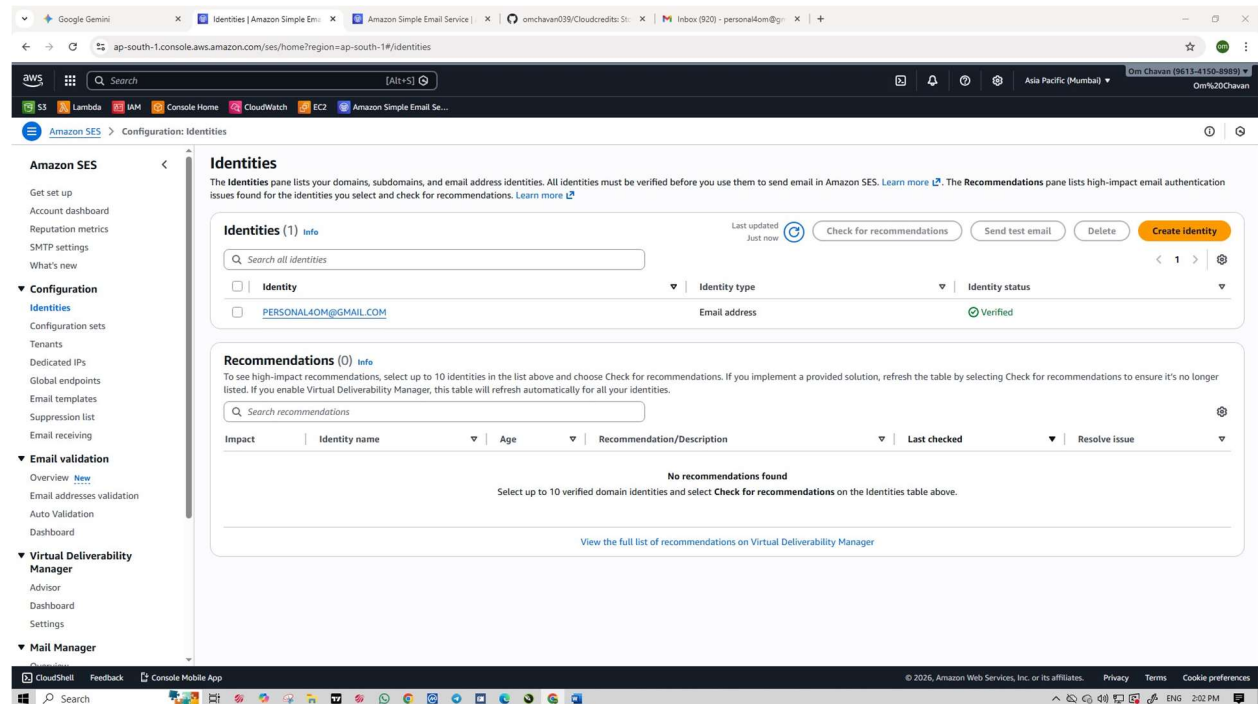
1. Introduction

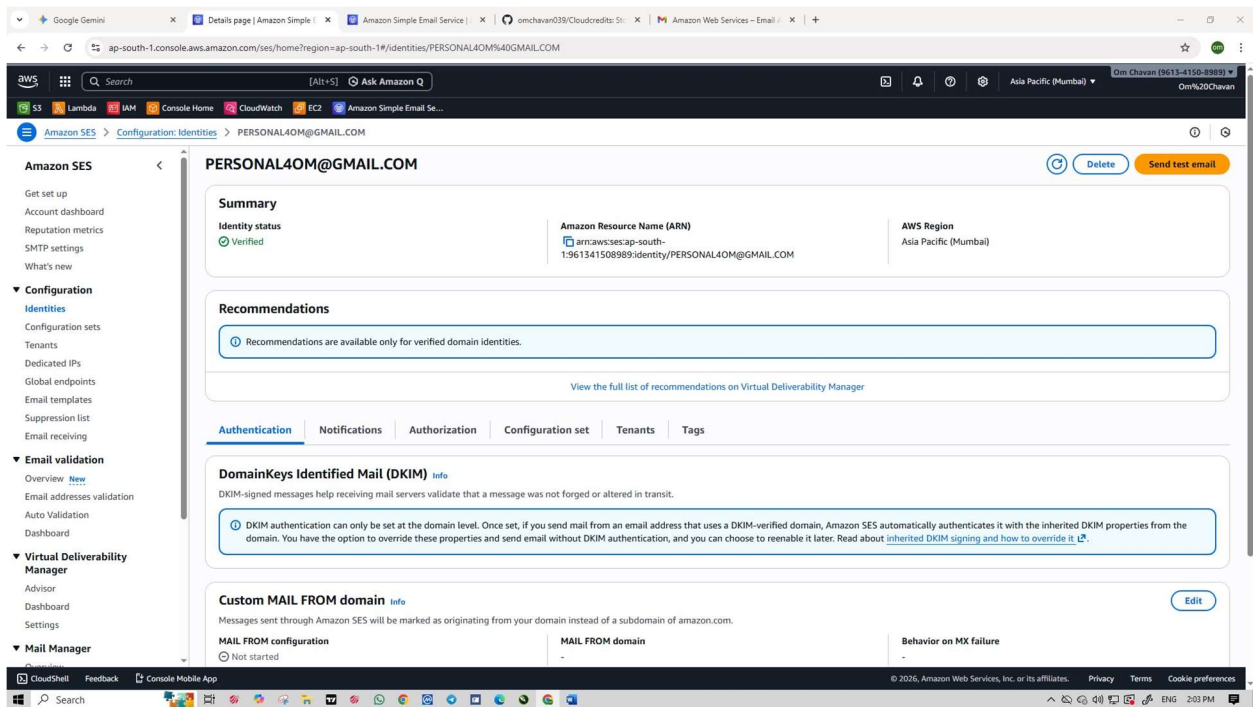
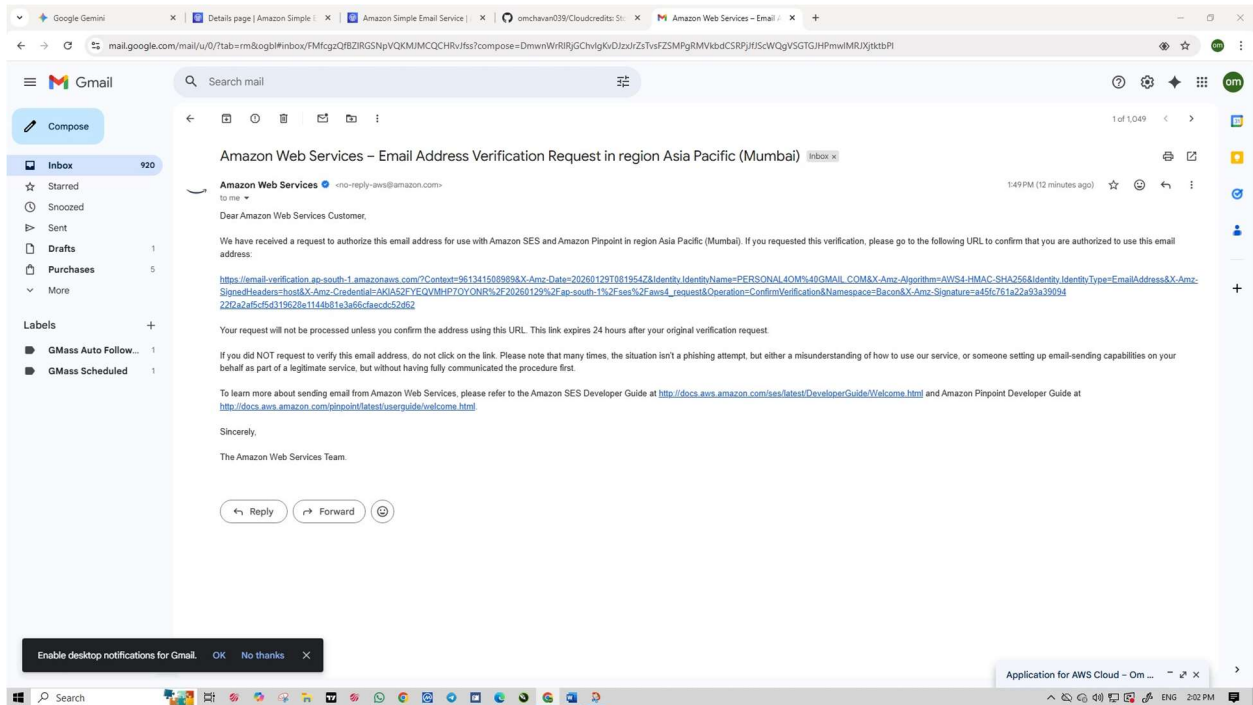
This project implements a serverless architecture to automate mass emailing. The system is triggered by an S3 event, processed by AWS Lambda, and delivered via Amazon SES.

2. Step-by-Step Configuration

Step 1: Amazon SES (Simple Email Service) Setup

- Navigated to the SES Console.
- Verified a sender email identity to ensure authorization.
- Confirmed the verification email sent by AWS.





Step 2: S3 Bucket Creation

- Created a unique S3 bucket named cloudcredits-project3
- Prepared a recipients.csv file with the following structure: OM MOHAN CHAVAN (Personal4om@gmail.com)

The screenshot displays the AWS Management Console interface, showing the process of creating an S3 bucket and viewing its contents.

Top Screenshot (Bucket Creation):

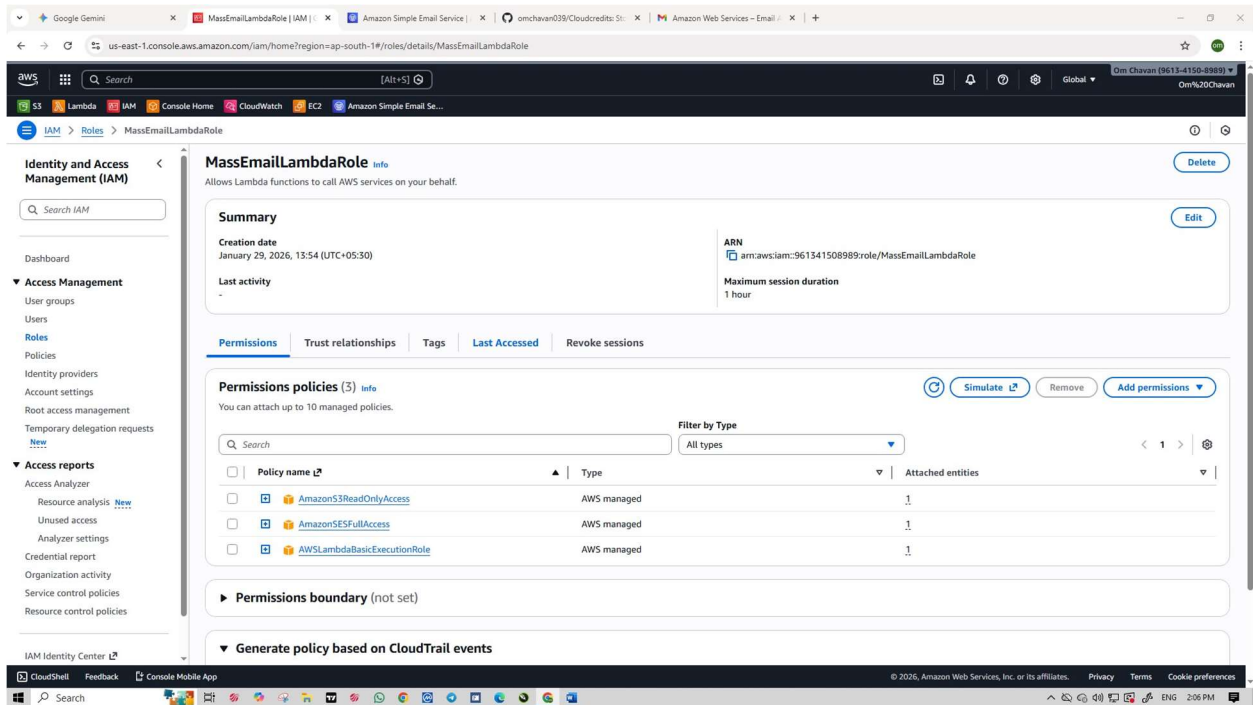
- Navigation:** The left sidebar shows the 'Amazon S3' service selected.
- General purpose buckets:** The main panel shows a list of buckets. The 'Create bucket' button is visible.
- Account snapshot:** A section on the right provides information about the account snapshot, including a 'View dashboard' link.
- External access summary:** A section on the right provides information about external access, including a 'View dashboard' link.

Bottom Screenshot (Bucket Contents):

- Navigation:** The left sidebar shows the 'Amazon S3' service selected.
- Bucket:** The main panel shows the 'cloudcredits-mass-email-data' bucket. The 'Objects' tab is selected.
- Objects:** A table lists the objects in the bucket. The object 'recipients.csv' is visible.
- Object Details:** The object 'recipients.csv' is selected, showing its details: Name, Type, Last modified, Size, and Storage class.

Step 3: IAM Role & Security

- Created an IAM Role for Lambda.
- Attached fine-grained policies: AmazonS3ReadOnlyAccess and AmazonSESEFullAccess.
- Ensured the "Trust Relationship" allows Lambda to assume the role.



Step 4: AWS Lambda Development

- Created a Python-based Lambda function.
- Added an S3 Trigger: All object create events.

- **Source Code:**

```
Python
```

```
import boto3
```

```
import csv
```

```
import io
```

```
s3 = boto3.client('s3')
```

```
ses = boto3.client('ses')
```

```
def lambda_handler(event, context):
```

```
    bucket = event['Records'][0]['s3']['bucket']['name']
```

```
    key = event['Records'][0]['s3']['object']['key']
```

```
    response = s3.get_object(Bucket=bucket, Key=key)
```

```
    content = response['Body'].read().decode('utf-8')
```

```
    reader = csv.DictReader(io.StringIO(content))
```

```
    for row in reader:
```

```
        ses.send_email(
```

```
            Source='your-verified-email@example.com',
```

```
            Destination={'ToAddresses': [row['Email']]},
```

```
            Message={
```

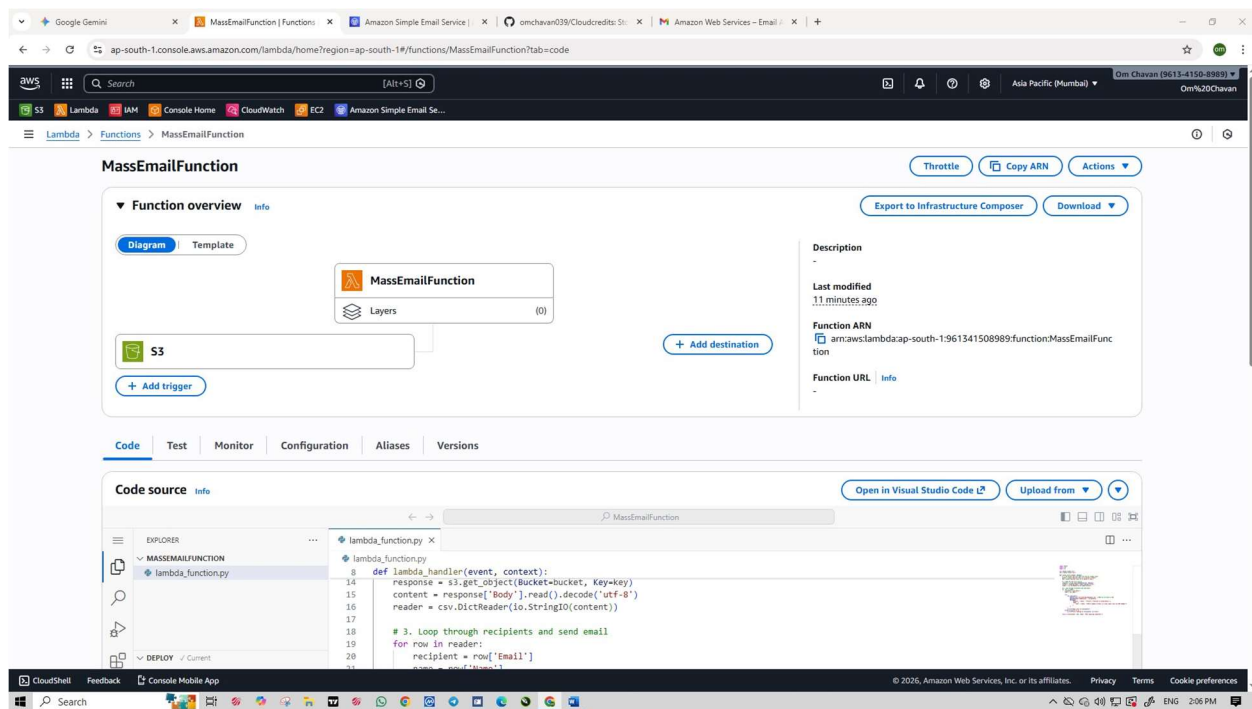
```
                'Subject': {'Data': 'Project 3 Submission'},
```

```
                'Body': {'Text': {'Data': f"Hello {row['Name']], Email sent via Lambda!"}}
```

```
            }
```

```
        )
```

```
    return {"status": "Success"}
```



Declaration: I hereby declare that this project was completed as part of the Cloudcredits Internship requirements.